

Postgres wait events analysis (a.k.a. Active Session History; ASH dashboard)

Goals and methodologies

This runbook outlines the steps for conducting a drill-down performance analysis, at the node level, from high-level view at the whole workload to individual queries (individual query IDs), based on wait event sampling using `pg_wait_sampling`.

The wait event centric approach is also known as:

- Active Session History in Oracle
- Performance Insights in AWS RDS
- Query Insights in GCP CloudSQL (where `pg_wait_sampling` is also available)

In those systems, this approach is often considered as the main one for workload performance analysis and troubleshooting.

The wait events analysis dashboard serves as a vital tool for database performance troubleshooting, making complex performance patterns accessible and actionable. While traditional monitoring might tell you that your database is running slowly, this dashboard helps pinpoint exactly why it's happening. By visualizing database load through the lens of wait events, it enables both database experts and application teams to:

- identify performance bottlenecks without needing to dive deep into database internals
- understand whether performance issues stem from CPU utilization, I/O operations, memory constraints, lock or lwlock contentions
- trace problematic wait events back to specific queries (identifying their `queryid` values – the same `queryid` that are used in other areas of query analysis such as based on `pg_stat_statements`, `pg_stat_kcache`, or logging including `auto_explain`).
- think of wait events as a queue at a busy restaurant - this dashboard shows you not just how long the line is, but why people are waiting (kitchen backup, seating limitations, or staff shortages) and which orders are causing the longest delays; this practical insight can help move from reactive firefighting to proactive performance management
- the ASH dashboard bridges the gap between observing performance problems and understanding their root causes, enabling faster and more accurate resolution of database performance issues

Originally, for each backend (session), Postgres exposes wait events in columns `wait_event_type` and `wait_event` in system view `pg_stat_activity` docs.

These events need to be sampled for analysis. With external sampling (e.g., dashboard involved Marginalia and `pg_stat_activity` sampling built in MR, the

frequency of sampling is not high, cannot exceed 1/sec, thus data is not precise. With `pg_wait_sampling`, the sampling is internal, with high frequency (default: 100/second, 10ms rate), which is then exported infrequently, but has much better coverage and precision of metrics, enabling wider spectrum of performance optimization and troubleshooting works.

Dashboards to be used

1. Postgres Wait sampling dashboard

Additionally, for further steps:

1. Postgres aggregated query performance analysis
2. Postgres single query performance analysis

Analysis steps

In all panels, the metric shown is “number of active sessions/backends that are busy with specified wait events / wait event types”. Please note that the presented values may differ from actual counts in `pg_stat_activity` due to several processing steps:

- Sampling performed by the `pg_wait_sampling` process
- Data aggregation during export
- Conversion formulas that estimate active sessions from sample counts

When the number of sessions shown on the Y-axis exceeds the number of available vCPUs AND the majority of wait events are CPU-intensive (non-IO-related), this indicates potential CPU capacity exhaustion. Such situations require immediate attention as they suggest the system is at or approaching its processing limits.

The dashboard has three panels, representing wait event analysis at three detailization levels:

1. Top level: wait event types for the whole workload on this node, without specific wait events (only types – the highest level of aggregation)
2. High level with ability to filter by `wait_event_type`, to see `wait_event` values for the specified wait event type (filtering here is optional)
3. Lower level with ability to filter by both `wait_event_type` and `wait_event` to see contribution of individual query IDs (both filters are optional)

This, scrolling the ASH dashboard from top to bottom and gradually applying filters (first, by wait event type, then by individual wait event), we can perform top-down wait event analysis, moving from node-level whole workload to individual queries.

Working at any level:

- Review the stacked graph showing all types of wait events (`Activity`, `BufferPin`, `Client`, `IO`, `IPC`, etc.)
- Pay special attention to the `CPU` or `Uncategorised` wait event types as they may indicate processing bottlenecks
- Note that different colors represent different wait event types for easy visual correlation

Step 1. Node wait events (ASH) overview, all wait events are visible and not filtered out

This panel shows the highest-level view of wait event analysis. Make sure you're looking at the Postgres cluster and the specific node you need to analyze – this can be changed at the top in many of the dashboards.

Step 2. Filter by wait event type. Events of given type are without query IDs

This panel shows the highest-level view of wait event analysis. Use `wait_event_type` filter at the very top of the dashboard to filter by wait event type and see what particular wait events are playing higher role inside the spikes for particular wait event type. See the official docs to understand the meanings of particular wait events (important: make sure you're checking out the docs for the Postgres version currently being used).

Step 3. Find query IDs contributing to given wait event type and wait event

This panel allows to identify the queries that are responsible for high values for specific wait events – we can do it finding the `queryid` values for those queries. To do that, make sure you use or not use filters on `wait_event_type` and `wait_event` in the top panel of the dashboard – these filters are optional and should be used to drill down in case if the nature of active session spikes is not trivial.

`queryid` values here are the same `queryid` that are used in other areas of query analysis such as based on `pg_stat_statements`, `pg_stat_kcache`, or logging including `auto_explain`.

Note that for some backends, you'll see textual value (e.g., `Postgres process - parallel worker`, `Postgres process - autovacuum worker`) instead of `queryid`. This gives ability to see the contribution of Postgres helper processes to load.